



Third Party Instruments with Dynacool / PPMS and outsourcing tasks by scripting

Concepts



- Scripting (WinWrap® Basic) – local solution
- LabVIEW™ – local or second PC
 - Communication through QDInstrument.dll (.NET)
- Telnet (python) based communication – second computer
- MultiPyVu – a python implementation
- Examples based on WWB scripting
- File analysis “on the fly” – by python. Useful warning monitor built from it
- CANopener – resource for low level elements
- General system remote access – TeamViewer



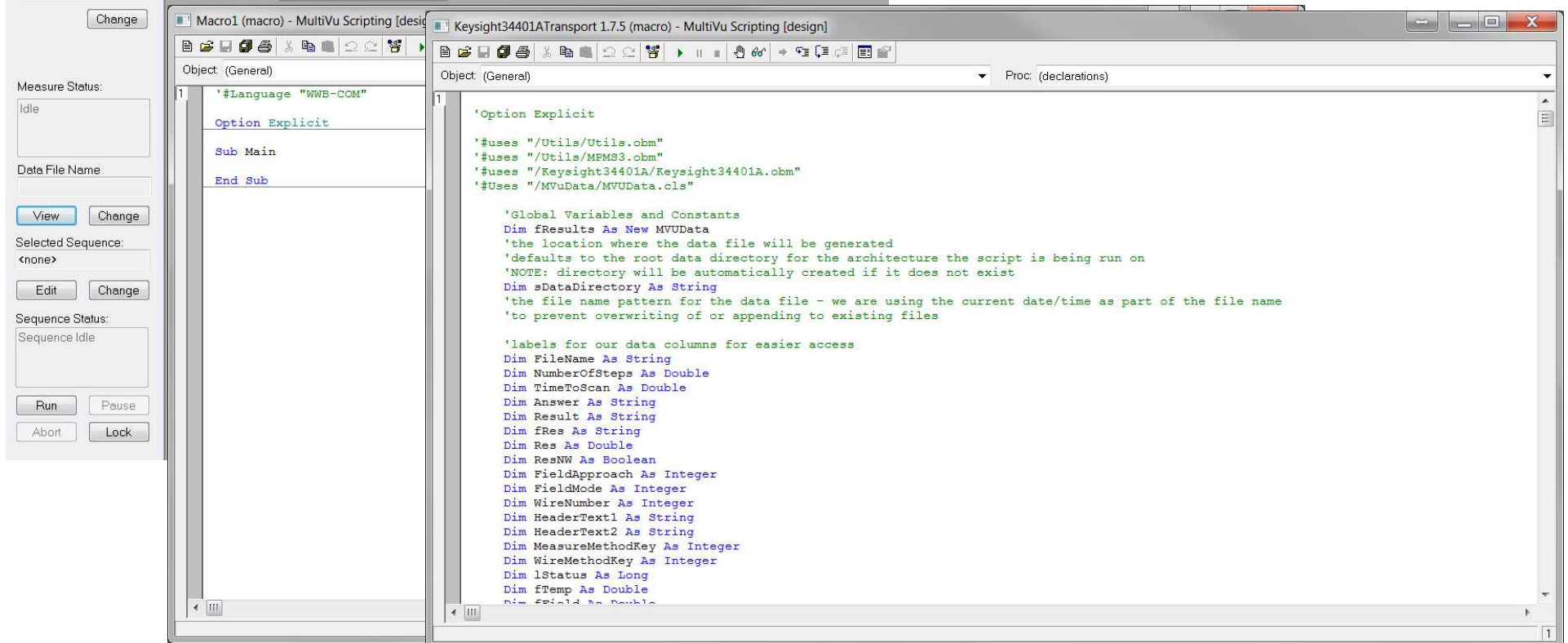
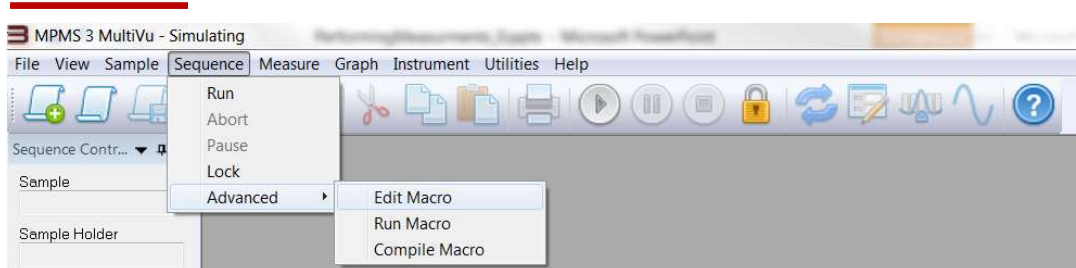
1. Scripting

Recommended for 3rd party applications



- MultiVu offers WinWrap Basic (Editor, Interpreter)
- Can use MV features in WWB code
 - E.g. T³ spacing in T scans (not available as standard command)
- Embed 3rd party device(s) in MV
 - Send / read / trigger
 - Communication with DLLs
 - Communication with ComPort
- Can create data files in MV format
- User dialog boxes
- Basic documentation:
- [https://www.winwrap.com/web2/basic/#!/ref/WWB-doc language_def.htm](https://www.winwrap.com/web2/basic/#!/ref/WWB-doc_language_def.htm)





WinWrap Basic - Scripting directly in MultiVu

Dim e As Double

```
Sub Mask
Begin Dialog UserDialog 360,259,"Convert Wavelength into Energy",.DialogFunc ' %GRID:10,7,1,1
Text 20,14,120,14,"Wavelength:",.Text1
TextBox 150,14,90,14,.wavelength
Text 60,98,90,14,"Frequency:",.Text4
TextBox 150,98,80,14,.frequencyBox
Text 250,98,90,14,"THz",.Text5
Text 60,119,90,14,"Omega",.Text6
TextBox 150,119,80,14,.omegaBox
Text 250,119,90,14,"Tera / sec",.Text7
CancelButton 190,224,90,21
Text 260,14,90,14,"nm",.Text9
Text 60,140,90,14,"Energy",.Text11
TextBox 150,140,80,14,.energy
Text 250,140,90,14,"eV",.Text12
PushButton 70,224,90,21,"OK",.btnOK
Text 60,161,90,14,"Wave number",.Text2
TextBox 150,161,80,14,.wavenumberBox
Text 250,161,90,14,"/cm",.Text3

End Dialog
Dim dlg As UserDialog

dlggrtn = Dialog(dlg)
If (dlggrtn = 0) Then End

Call Mask
End
'Call CalculateAcceleration
'DlgText("MaxVelBox",CStr(maxvelocity))
'DlgText("omegaBox",CStr(omega))
'DlgText("frequencyBox",CStr(frequency))
'DlgText("acceleration",CStr(acceleration))
'DlgText("gravity",CStr(gravityvalue))
End Sub
```

```
Function DialogFunc(DlgItem$, Action%, SuppValue&) As Boolean
```

```
Dim bValid As Boolean
```

```
'Case 1 Dialog box initialization
```



Notes about WinWrap - scripting

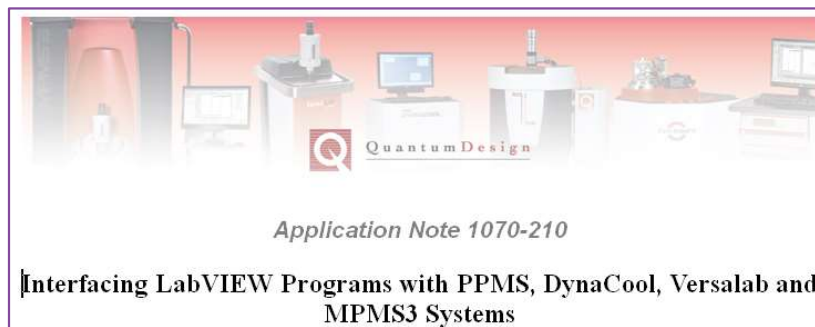
- Scripts are located in the directory `c:\QDDynacool*\macros`
- App Note 1070-209 (<https://www.qdusa.com/siteDocs/appNotes/1070-209.pdf>) „Getting started with Visual Basic scripting in MV“
- Resources (https://www.qdusa.com/siteDocs/appNotes/Macros_QD_Library_Oct_2015.zip) with examples for
 - Agilent E3644A DC Power Supply
 - Agilent LCR meter E4980A
 - Keithley 2182A and 6220
 - Stanford Research SR830 lock-in

* or PPMS, Versalab, SquidVSM, OptiCool



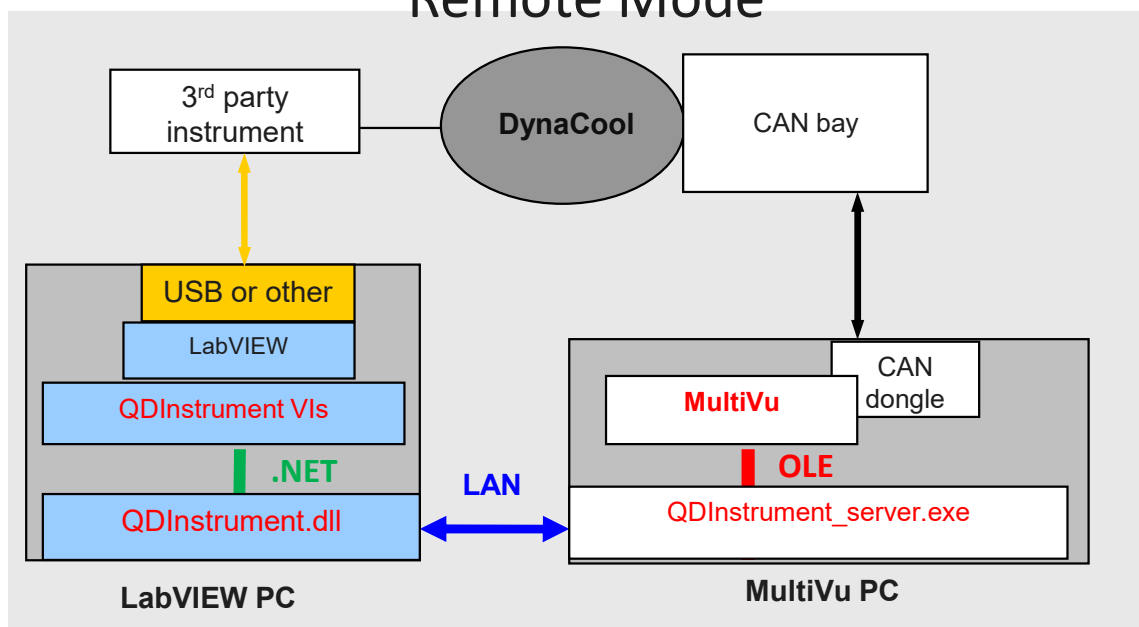
-
- internal to MultiVu
 - *Sequence > Advanced > Edit Macro*
 - examples exist
 - input/output
 - data file handling
 - .OBM files there for
 - Lock-ins
 - bridges
 - LCR meters...
 - library is growing
 - NOTE: WinWrap only addresses one GPIB master device
 - Operation of GPIB devices requires addition of USB ↔ NI-GPIB adapter (available at QDESC, part number: QDS-MA1) to other instruments than PPMS

2.

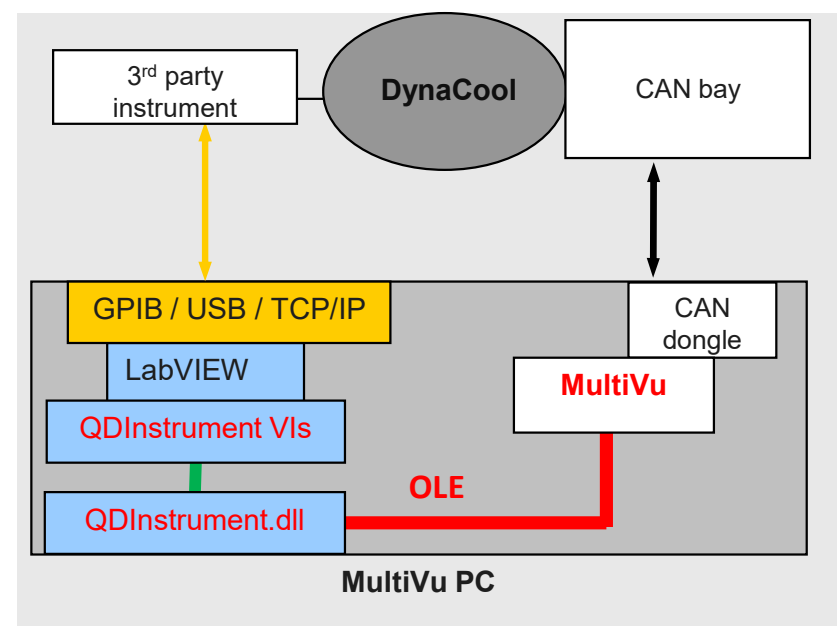


<https://www.qdusa.com/siteDocs/appNotes/1070-202.pdf>

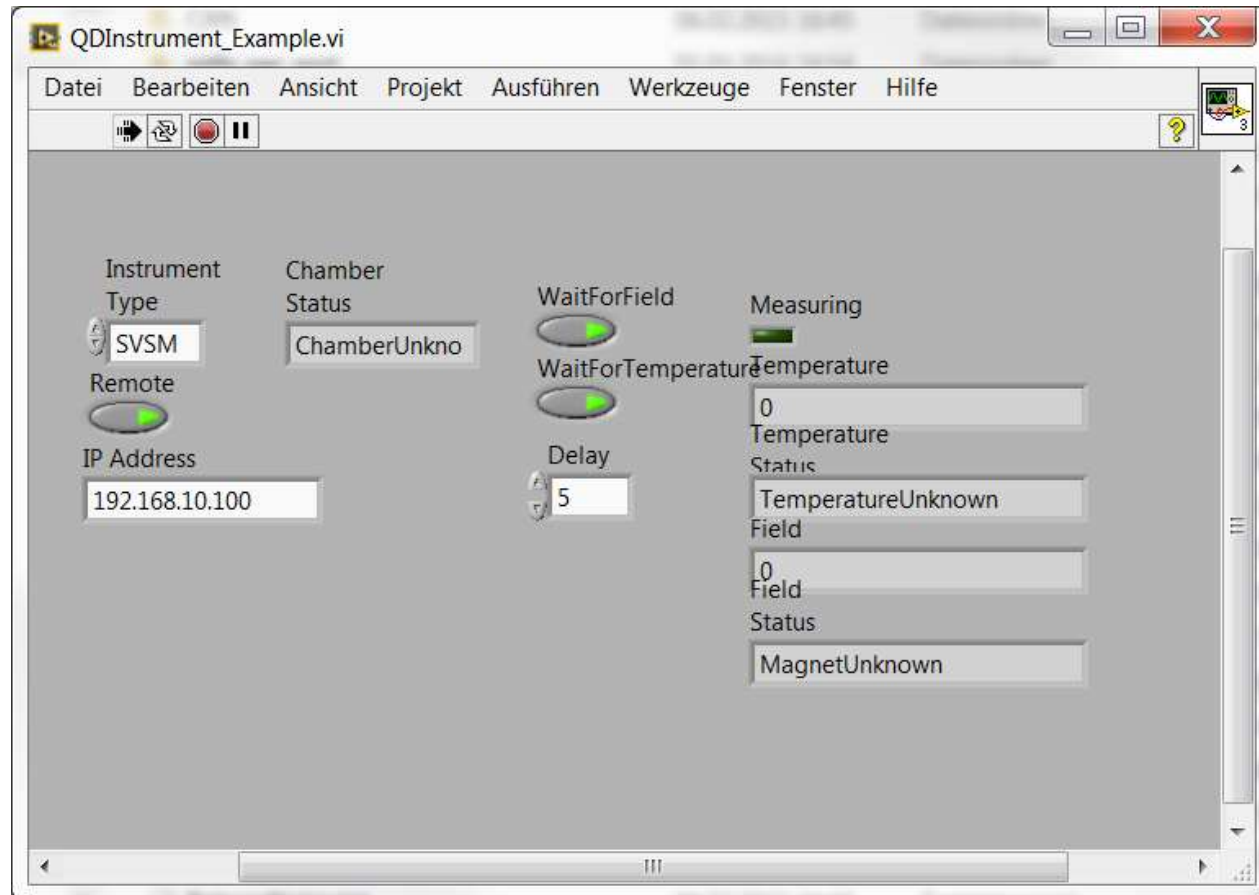
Remote Mode



Local Mode



LabVIEW example (<https://www.qdusa.com/pharos/view.php?fDocumentId=2704>)



LabVIEW

- Software package (<https://www.qdusa.com/pharos/view.php?fDocumentId=2704>) from QD webpage
- Run instrument server on DynaCool computer in parallel to running MV-task
- Connect to service by LabVIEW accessible QDinstrument.dll (that can become used by YOUR software)
- Example code to run QD Instrument(s) is part of the package
- LabVIEW software requires licence

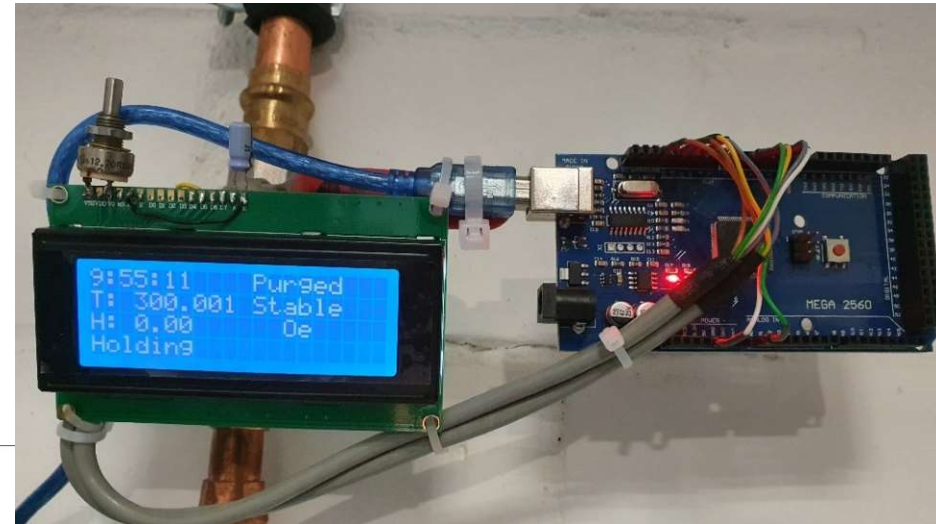


.Net – lower level entry point

- Use methods of QDInstrument.dll by your .Net capable programming language, e.g. VB, C#, python
 - QDInstrument.dll is part of the QD LabView package (free):
- resources from pharos (.Net examples)
 - <https://www.qdusa.com/pharos/view.php?fDocumentId=2704>



- C#- code of monitoring software based on .Net
- Code runs in parallel to active MV instance



DynaCool MultiVu

File View Sample Sequence Measure Graph Instrument Utilities Help

Selected Sequence: Ni_std.seq

Sequence Status: Sequence Idle

Run Pause Abort Lock

PPMS DYNACool

Dynacool-Monitor parsing datagrams to Com# 12

Basic SDO

Temperature: Temp. (K) 200.000 Status (Code) 5

Field: Field (Oe) 0.00 Status (Code) 4

Chamber: Status (Code) 7

Monitor sends info to serial port of PC

Sequence Idle	298.00 K, Stable	123.46 Oe, Holding	760 Torr	Tank: 760 Torr
Seq: <none>	Set: 298.00 K	Set: 123.46 Oe	Sealed	Warm
<none>	20.00 K/min, Fast Settle	0.00 Oe/sec, Linear		Magnet: 0.00 K

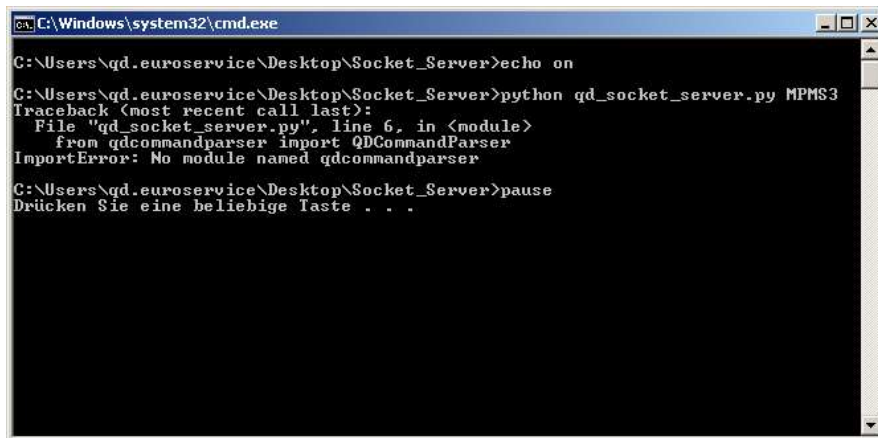
3. Socket Server / Telnet

- Telnet is a TCP/IP protocol
- Telnet can work on nearly any computer (Unix, Linux, MacOS, Win,)
- Can be used to control PPMS parameters (T, H, chamber)
 - Send and read
- 2nd computer controls 3rd party unit and communicates with MV PC by Telnet
- Telnet = SocketService (<https://www.qdusa.com/pharos/view.php?fDocumentId=2703>) in QD world
 - Requires Python interpreter (free)



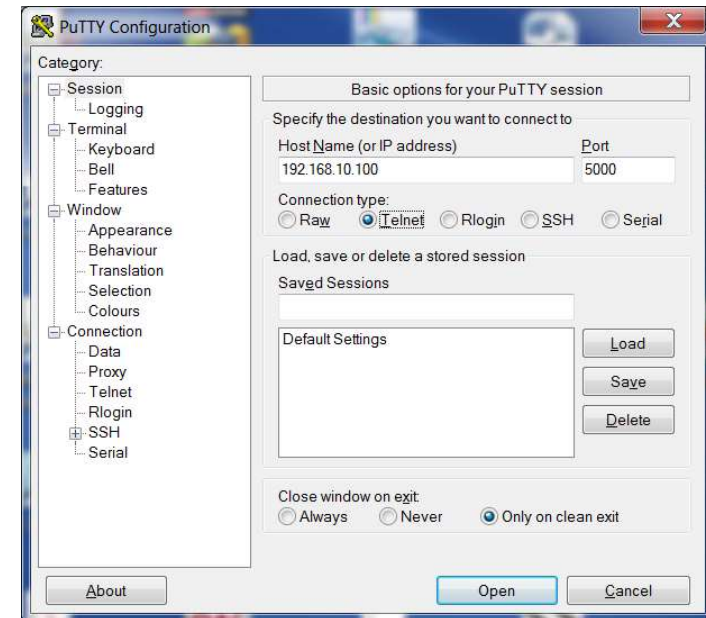
Telnet session to be started by batch file on QD computer Telnet client (e.g. PuTTY) to be started (embedded) on master.

- Local system running MultiVu:



```
C:\Windows\system32\cmd.exe
C:\Users\qd.euroservice\Desktop\Socket_Server>echo on
C:\Users\qd.euroservice\Desktop\Socket_Server>python qd_socket_server.py MPMS3
Traceback (most recent call last):
  File "qd_socket_server.py", line 6, in <module>
    from qdcommandparser import QDCommandParser
ImportError: No module named qdcommandparser
C:\Users\qd.euroservice\Desktop\Socket_Server>pause
Drücken Sie eine beliebige Taste . . .
```

- Remote System:



Telnet Server based access

- MV computer needs Python interpreter. Version 2.7 or 3.4 or higher.
<https://www.python.org> (an IDE is not a must here)
- Server package required from Pharos:
<https://www.qdusa.com/pharos/view.php?fDocumentId=2703>
- Run StartServerDynaCool.bat in environment of python. Ensure system(s) port #5000 open (firewall)
- Access from outside by telnet protocol (to be embedded in your third party sw application, e.g . matlab). A PuTTY  should work well for first try.
- Consult readme.txt for usage details, e.g.: (→next slide)

TEMP?

This returns something like "0, 300.0, 1" where 0 is a return code, 300.0 is the temperature, and 1 is the temperature status (stable)

Other Commands:

TEMP?

return_code, temperature, status

TEMP temperature_setpoint, rate, mode, return_code

FIELD?

return_code, field, status

FIELD field_setpoint, rate, approach_mode, end_mode, return_code

CHAMBER? return_code, state

CHAMBER code, return_code

The following commands do not interact with MultiVu, but are for the server:

CLOSE This closes the socket connection, leaving the server running (other programs can still connect).

EXIT This causes the server to exit. No further connections are possible without restarting the server.



python package from QD

- App note <https://qdusa.com/siteDocs/appNotes/1070-220.pdf>

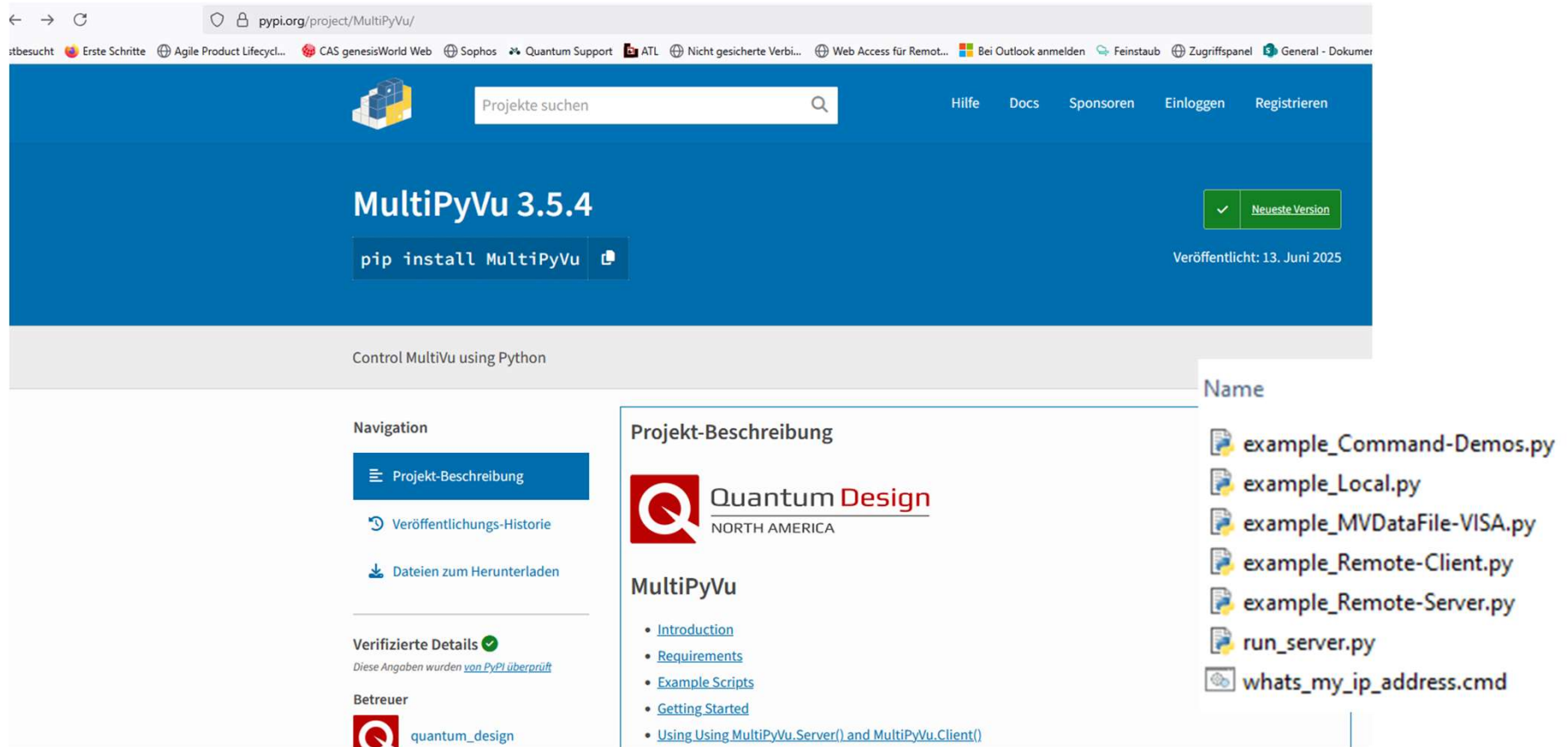
Platform: PPMS, DynaCool, VersaLab, MPMS-3, OptiCool



MultiPyVu: Controlling QD Cryostats Using Python



MultiPyVu, <https://pypi.org/project/MultiPyVu/>



The screenshot shows the PyPI page for MultiPyVu 3.5.4. The page has a blue header with the project name and version, a search bar, and navigation links. Below the header, there's a section for the project description, including a logo for Quantum Design North America and a list of links for Introduction, Requirements, Example Scripts, Getting Started, and Using MultiPyVu.Server() and MultiPyVu.Client(). On the right, there's a list of example files and scripts, including example_Command-Demos.py, example_Local.py, example_MVDataFile-VISA.py, example_Remote-Client.py, example_Remote-Server.py, run_server.py, and whats_my_ip_address.cmd. The page also includes a navigation sidebar with links to Projekt-Beschreibung, Veröffentlichungs-Historie, and Dateien zum Herunterladen.

Note support from QD's Application-Web-Page: https://www.qdusa.com/support/applications_team.html



Versions $\geq 3.5.4$ supporting a rotator !



Scripting

Recommended for 3rd party applications



- MultiVu offers WinWrap Basic (Editor, Interpreter)
- Can use MV features in WW code
 - E.g. T³ spacing in T scans (not available as standard command)
- Embed 3rd party unit(s) in MV
 - Send / read / trigger
- Can create data files in MV format
- Can be used for user dialog boxes
- Can communicate with DLLs
- Basic documentation:
- <https://www.winwrap.com/web2/>

4. Examples based on WWB scripting

- a) Keysight 34401A based resistivity measurements (GPIB)
- b) Tuneable lightsource (USB, DLL)
- c) Wavelength $\leftrightarrow$ energy converter (software tool)
- d) Van der Pauw measurements (Com-Port, DLL)
- e) Switch (multiplex) one 4-point device among the three channels (Com-Port, DLL)
- f) Stanford Research 810 Lock-In amplifier (GPIB)
- g) Call python-script by WWB code
- h) Call executable and wait until finished by WWB code
- i) Usage of serial com port
- j) Lakeshore M81 (SSM) & DynaCool

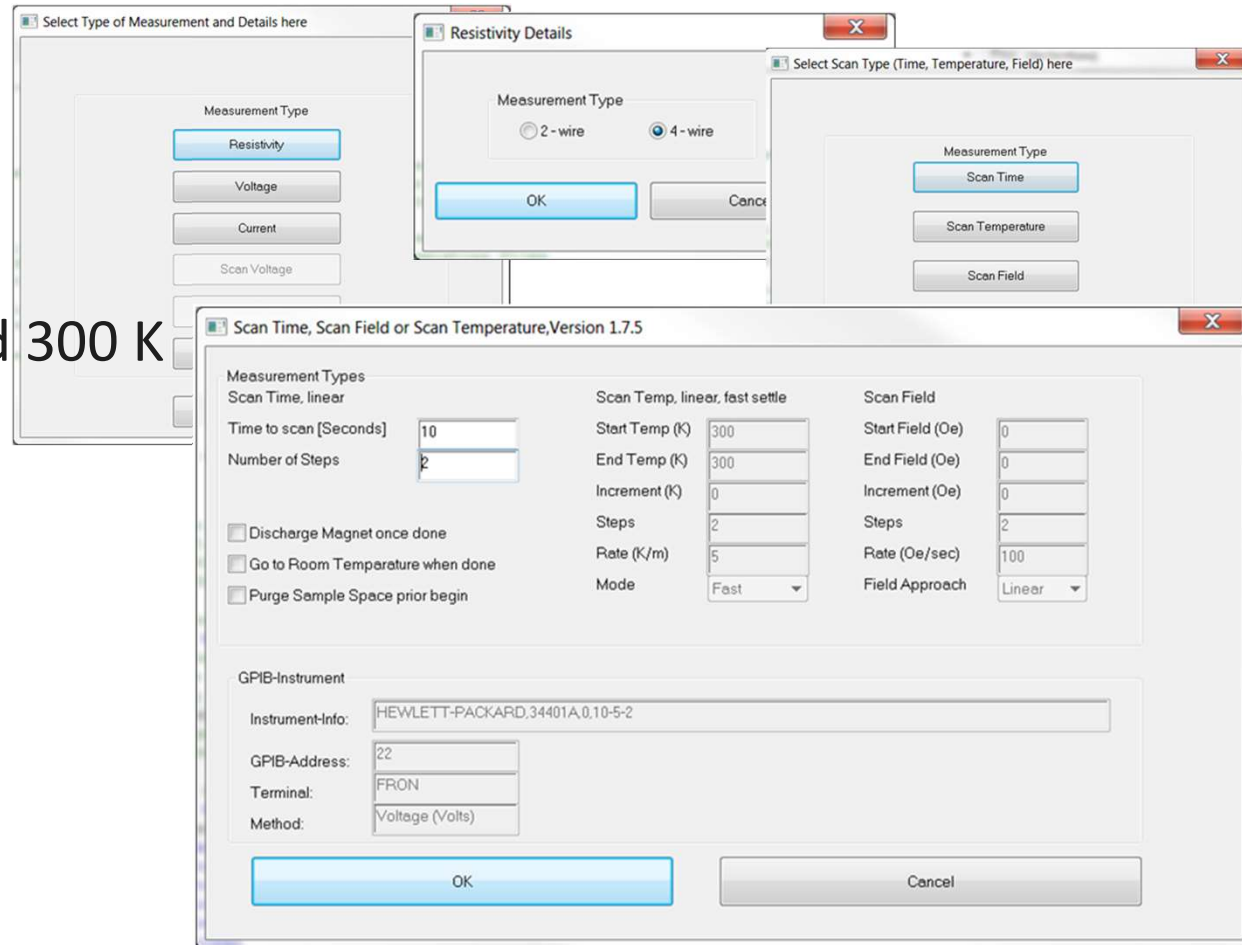
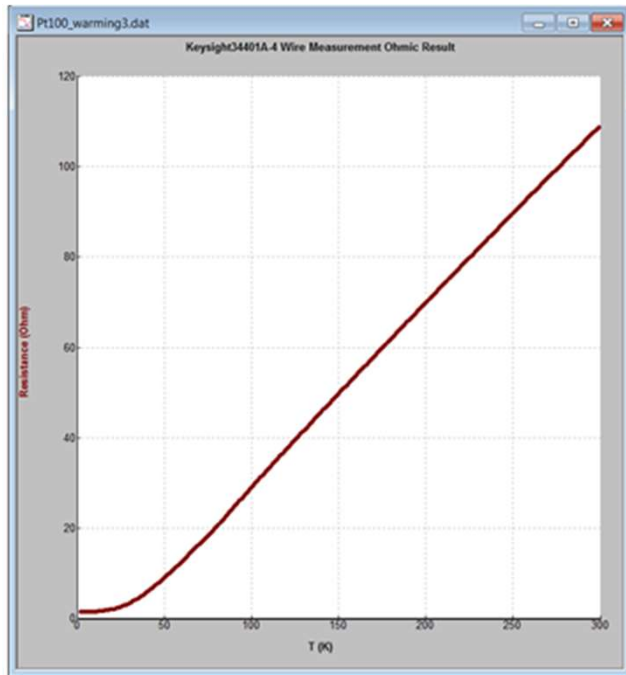


a) Resistivity with an external bridge (MPMS3 & DynaCool)



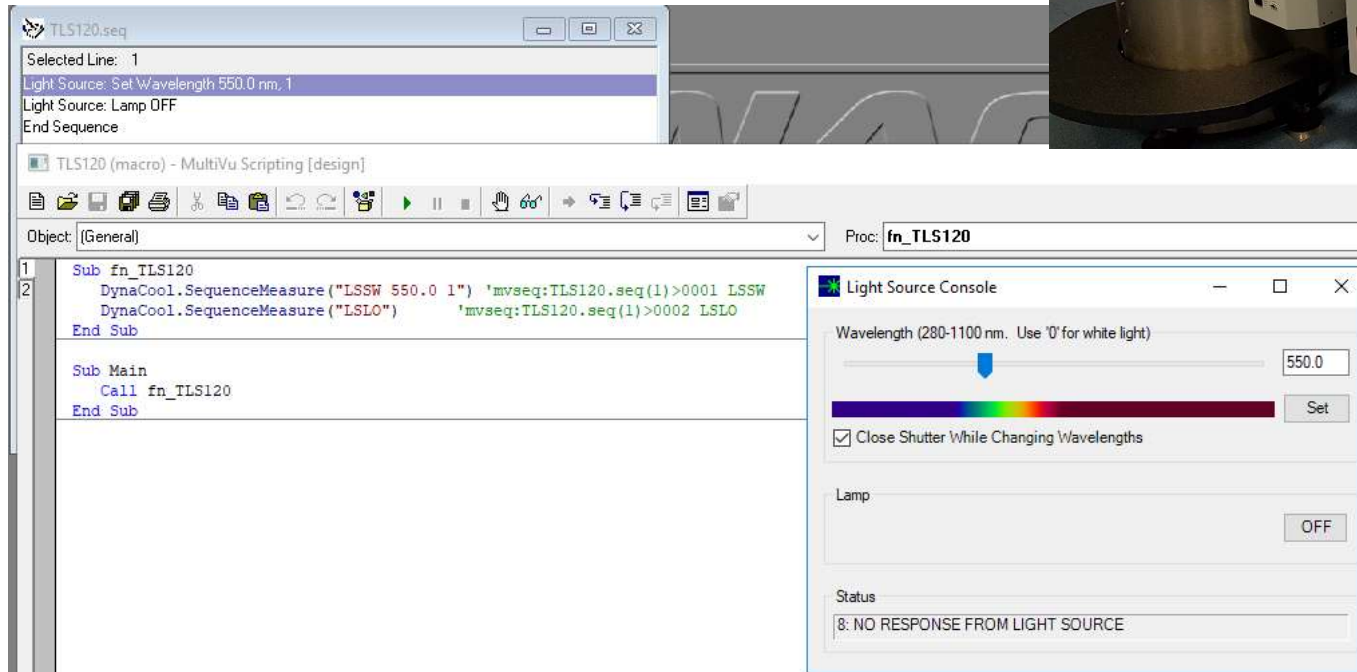
Resistivity

- Resistance of a Pt100 sensor
- in Zero field between 2 K and 300 K

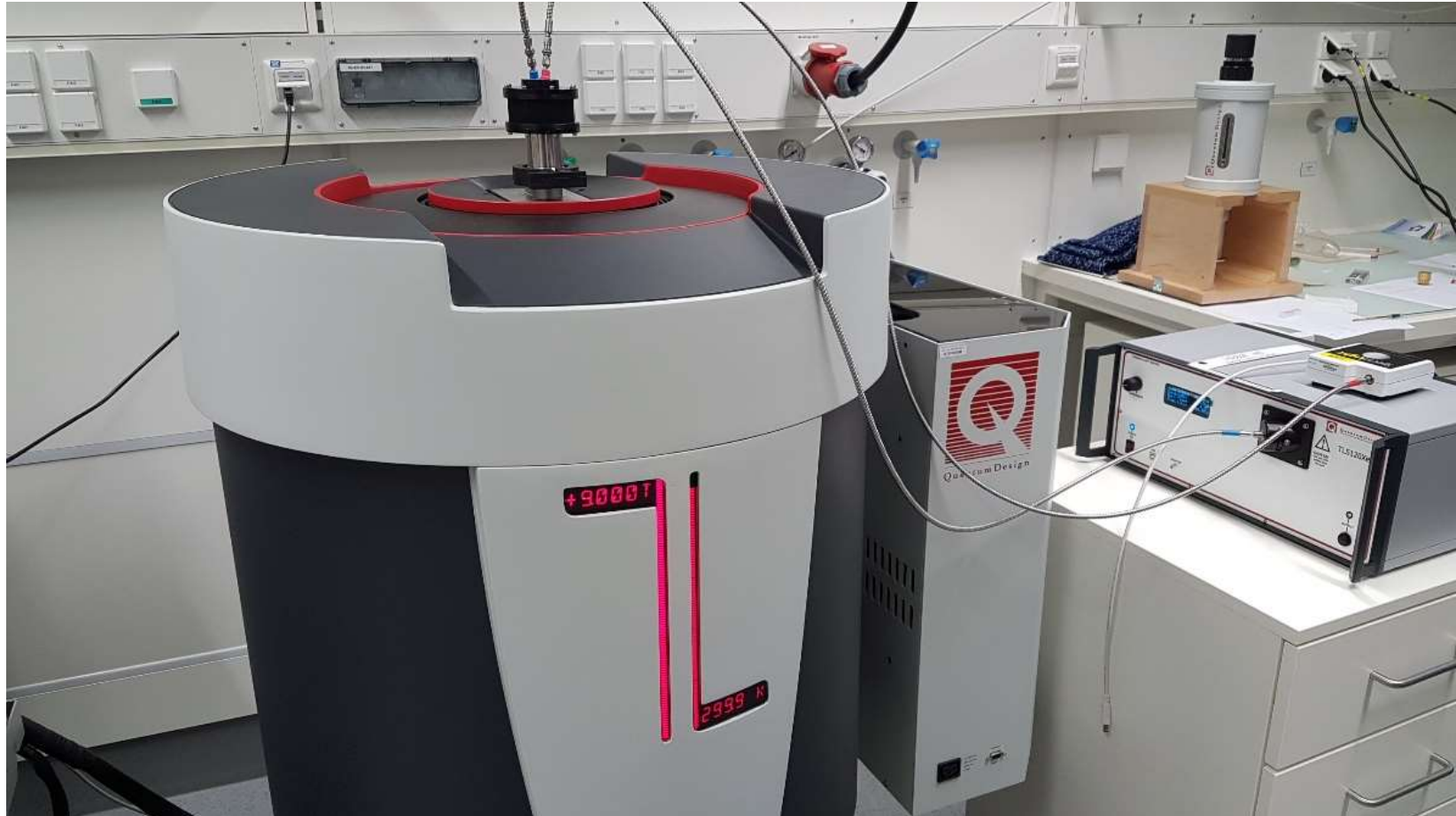


b) Tunable Lightsource (TLS120)

- 20 nm Bandwidth
- 280 .. 1100 central wl



Light-Source (USB), MFP, Spectrometer (USB)



c) Wavelength to energy converter on MV by scripting

Convert Wavelength into Energy

Wavelength: 632.8 nm

Frequency: 473.755 THz

Omega: 2976.693 Tera / sec

Energy: 1.959 eV

Wave number: 15802.781 /cm

OK Cancel

```
Dim e As Double

Sub Mask
Begin Dialog UserDialog 360,259,"Convert Wavelength into Energy",.DialogFunc ' %GRID:10,7,1,1
Text 20,14,120,14,"Wavelength:",.Text1
TextBox 150,14,90,14,.wavelength
Text 60,98,90,14,"Frequency:",.Text4
TextBox 150,98,80,14,.frequencyBox
Text 250,98,90,14,"THz",.Text5
Text 60,119,90,14,"Omega",.Text6
TextBox 150,119,80,14,.omegaBox
Text 250,119,90,14,"Tera / sec",.Text7
CancelButton 190,224,90,21
Text 260,14,90,14,"nm",.Text9
Text 60,140,90,14,"Energy",.Text11
TextBox 150,140,80,14,.energy
Text 250,140,90,14,"eV",.Text12
PushButton 70,224,90,21,"OK",.btnOK
Text 60,161,90,14,"Wave number",.Text2
TextBox 150,161,80,14,.wavenumberBox
Text 250,161,90,14,"/cm",.Text3

End Dialog

Dim dlg As UserDialog

dlggrtn = Dialog(dlg)
If (dlggrtn = 0) Then End

Call Mask
End

'Call CalculateAcceleration
'DlgText ("MaxVelBox",CStr(maxvelocity))
'DlgText ("omegaBox",CStr(omega))
'DlgText ("frequencyBox",CStr(frequency))
'DlgText ("acceleration",CStr(acceleration))
```

Converter Code for testing – copy, name *.bas and play ...

```
"Language" = "VB6 COM"

Option Explicit

Dim frequency As Double
Dim energy As Double
Dim v As Double
Dim wavelength As Double
Dim scanlength As Double
Dim digits As Integer
Dim energy As Double
Dim wavenumber As Double
Dim h As Double
Dim e As Double

Sub Main

    Begin Dialog UserDialog 800,510 "Convert Wavelength into Energy",DialogTitle "HGRO 03.1.1"

        Text 20,10,100,14 "Wavelength",Text1
        TextBox 100,10,100,14 wavelenght

        Text 40,160,14 "Frequency",Text4
        TextBox 100,160,14 frequencybox

        Text 20,300,14 "Plan",Text5
        Text 40,310,14 "Energy",Text6
        TextBox 100,310,14 energybox

        Text 20,510,14 "Plan / hc",Text7
        CancelButton 100,520,15

        Text 20,540,14 "hc",Text9
        Text 40,540,14 "Energy",Text11
        TextBox 100,540,14 energy

        Text 20,580,14 "hv",Text12
        PushButton 10,510,15,15 "OK" 600,510

        TextBox 100,580,14 wavenumberbox

    End Dialog

End Sub
```



End Dialog

Dim dlg As UserDialog

dlgfrm = Dialog(dlg)

If (dlgfrm = 0) Then End

Call Mask

End

'Call CalculateAcceleration

'DlgText("MaxVelBox",CStr(maxvelocity))

'DlgText("omegaBox",CStr(omega))

'DlgText("frequencyBox",CStr(frequency))

'DlgText("acceleration",CStr(acceleration))

'DlgText("gravity",CStr(gravityvalue))

End Sub

Function DialogFunc(DlgItem#, Action#, SuppressMask) As Boolean

Dim bRetVal As Boolean

'Case 1 Dialog box initialization

'Case 2 Value changing or button pressed

'Case 3 TextBox or ComboBox text changed

'Case 4 Focus changed

'Case 5 Title

'Case 6 Function key

Select Case Action#

Case 1 'Populates defaults

DlgDisable("MaxVelBox",False) 'gray

DlgDisable("omega",False) 'gray

DlgDisable("omegaBox",False) 'gray

DlgDisable("frequencyBox",False) 'gray

DlgDisable("waveNumberBox",False) 'gray

DlgText("wavelength",1/12.8)

For MeasureDetails

'Case 2 'Value changing or button pressed

'Case 3 'Textbox or ComboBox text changed

'Case 4 'Focus changed

'calculate delta Temp, populate box in Dialog

Case 5 ' On Idle

' validate mask input (en/disable OK button accordingly) - some logic probing - more might be necessary

```
Should = False

If (IsNumeric(DlgText("wavelength"))) Then

    If (CDbl(DlgText("wavelength")) > 0) Then

        Valid = True

    End If

End If

If Not (IsNumeric(DlgText("TimeBox"))) Then

    '

    WWS.MsgBox("Enter Integer Number >= 0", vbCritical, "Warning !")

End If

If Not (IsNumeric(DlgText("ScaleLengthBox"))) Then

    '

    WWS.MsgBox("Enter Decimal Number > 0", vbCritical, "Warning !")

End If

If MeasureDetailSubstitution = 0 Then

    'Call In_TypeDialog

End If

If Should Then

    wavelength = CDbl(DlgText("wavelength"))

    Call CalculateEnergy

    DlgText("MaxVelBox", CStr(Format(maxvelocity, "f.000")))

    DlgText("omegaBox", CStr(Format(omega/(1e12, "f.000"))))

    DlgText("frequencyBox", CStr(Format(frequency/(1e12, "f.000"))))

    DlgText("energy", CStr(Format(energy, "f.000")))

    DlgText("wavenumberBox", CStr(Format(wavenumber, "f.000")))
```

Else

DlgDisable("btnOK",False)

End If

' limit the number of calls to 10 per second and make sure we keep firing idle events

Wait(1)

' auto-close dialog box after 5min so the script will continue when left unattended

If (StdLib.GetTimeStamp() - DialogTimer > 300) Then

DlgShow(1)

End If

DialogFunc = True

End Select

End Function

Sub CalculateEnergy

frequency = c / (wavelength * 1e-9) ' frequency

omega = 2 * 3.1415927 * frequency 'radofreque

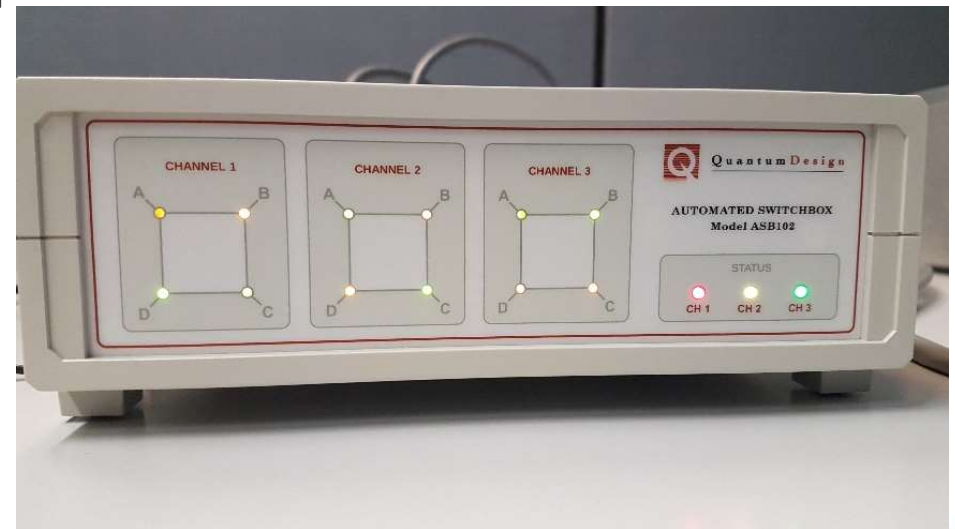
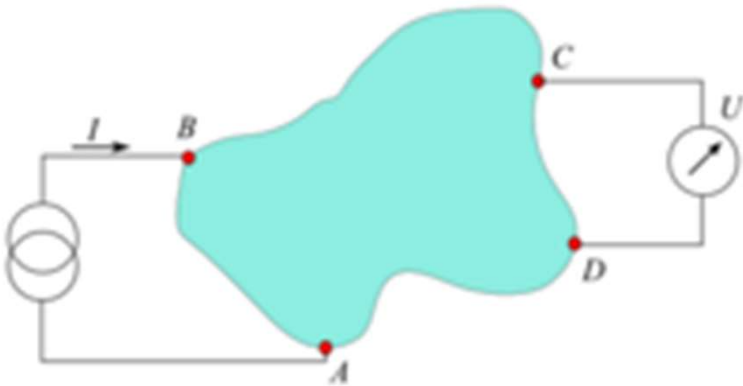
energy = h * frequency / e 'energy, eV

wavenumber = 1/wavelength*1e7'wavenumber = reciprocal wavelength (1/cm)

End Sub

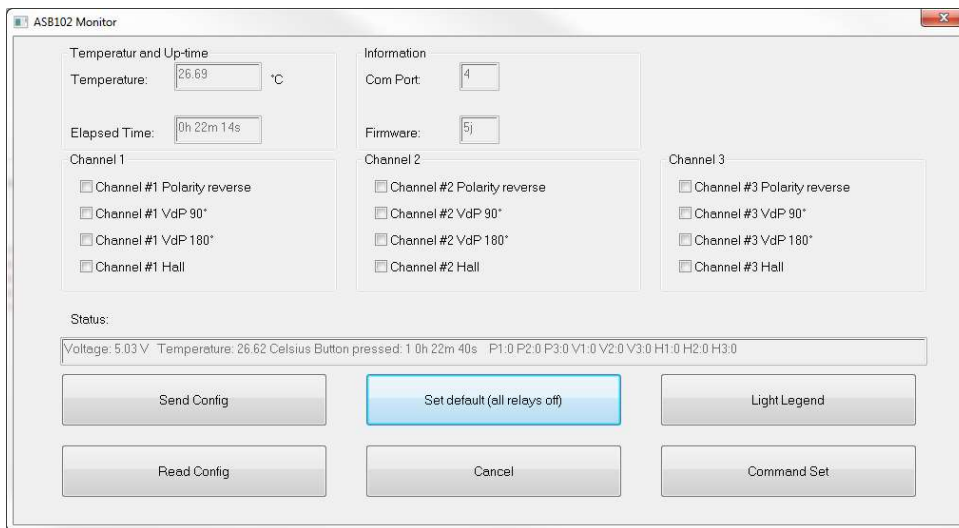
d) Van der Pauw switch box ASB102

- The box supports 3 channels for 4-wired samples
- 3 major permutations supported:
 - Reverting polarity of $I+$ vs. $I-$ (e.g. useful measuring semiconductors by just an unipolar DC-source)
 - Switching between standard and Van der Pauw pattern (90° , 180° , 270°)
 - Switching between standard and Hall setup

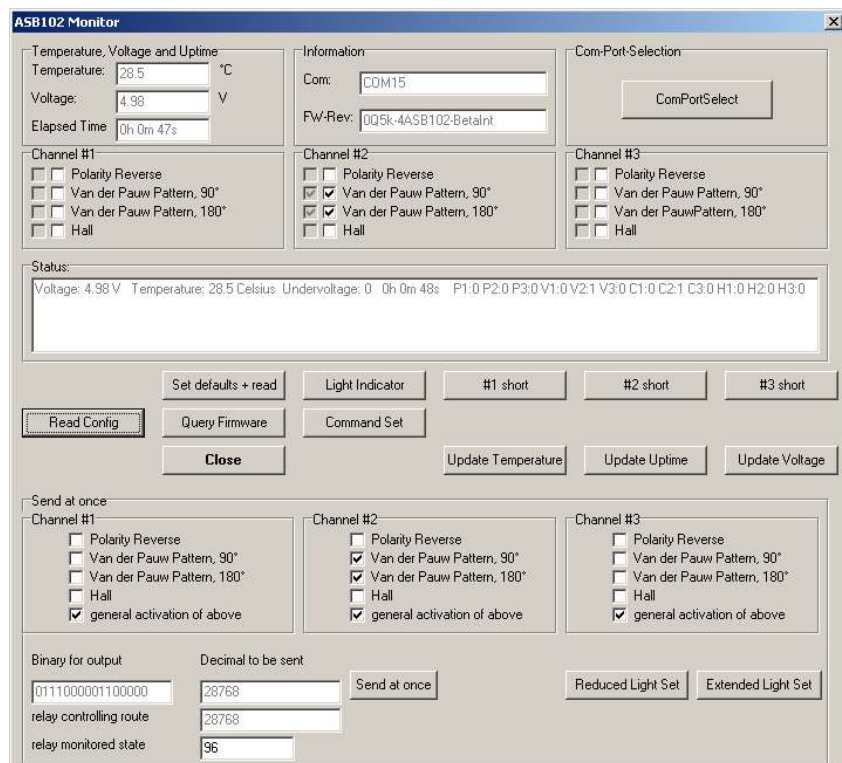


ASB102 controlled by scripting or integration into MultiVu

- MV-option SW: <https://www.qdusa.com/pharos/view.php?fDocumentId=4190>



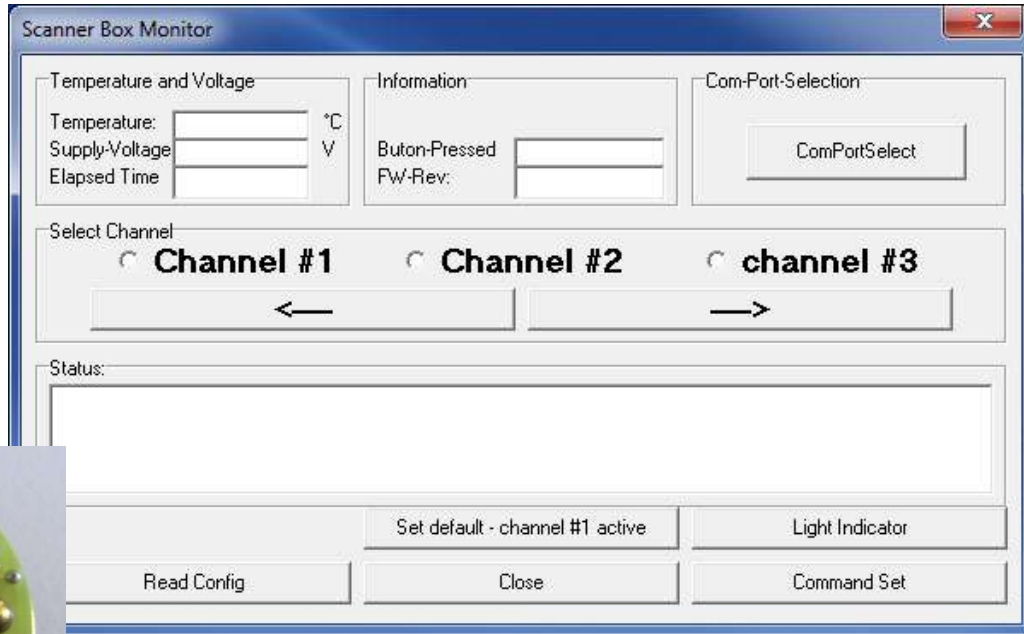
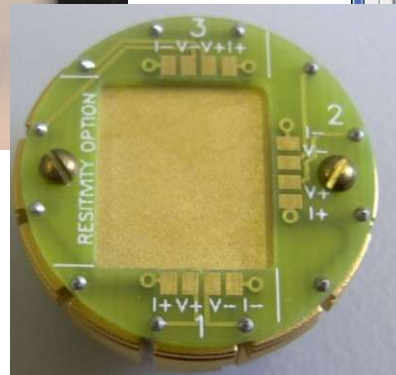
User dialog mask based on MV scripting



ASB102 monitor

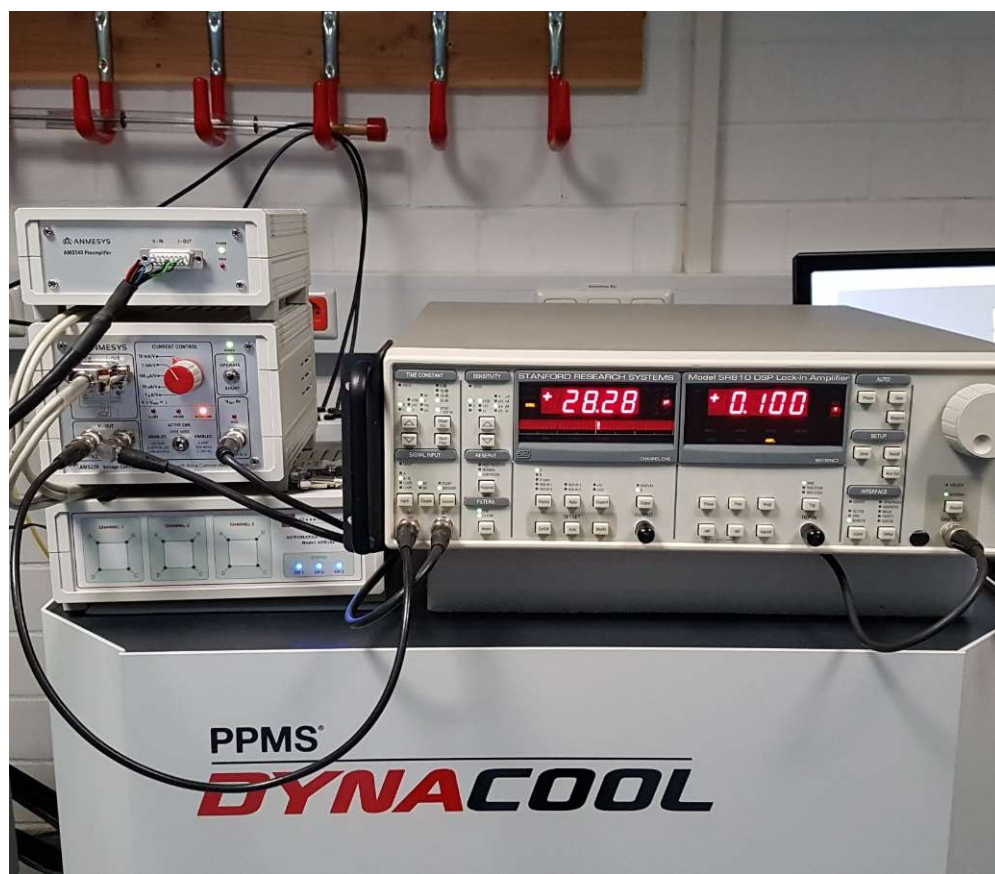


e) PPMS channel scanner – matching resistivity puck

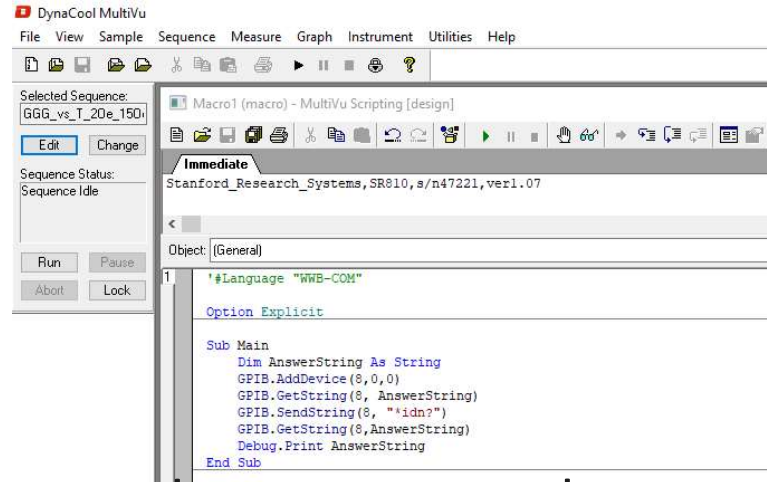


Homebrew device, Arduino nano μ C based

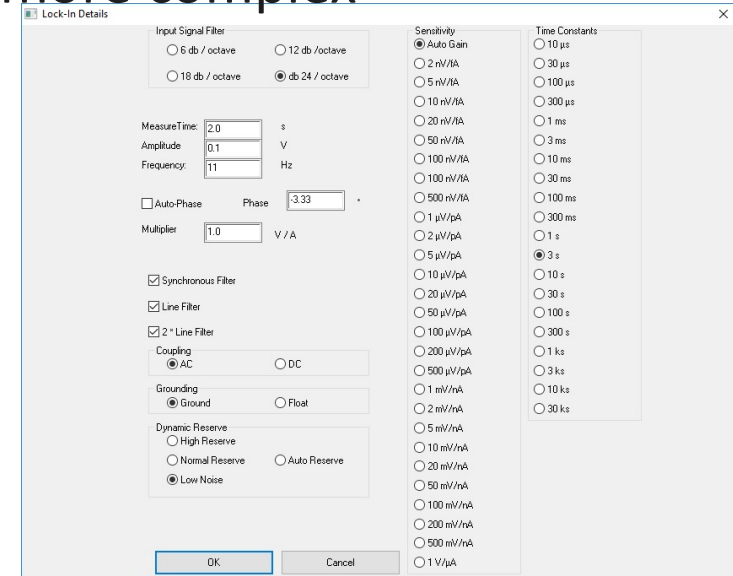
f) LIA (Lock-In Amplifier)



We start simple ...



... and go more complex



g) Call python script by WWB

[RunShellScriptAbsolutePath.BAS]

'#Language "WWB-COM"

Option Explicit

Sub Main

Shell("/Users/xxxxxxx/AppData/Local/Programs/Python/Python37-32/python.exe ./PythonCalledFromMV.py", vbNormalFocus)

End Sub

Ensure path matching your environment. Adapt !

[PythonCalledFromMV.py]

print("Python code called from MultiVu !")

input("Press Enter to continue...")



h) Call executable (shell script) by WWB

```
'#Language "WWB-COM"
```

```
Option Explicit
```

```
Private Const WAIT_INFINITE = -1&
```

```
Private Const SYNCHRONIZE = &H100000
```

```
Private Declare Function OpenProcess Lib "kernel32" (ByVal dwDesiredAccess As Long, ByVal bInheritHandle As Long, ByVal dwProcessId As Long) As Long
```

```
Private Declare Function CloseHandle Lib "kernel32" (ByVal hObject As Long) As Long
```

```
Private Declare Function WaitForSingleObject Lib "kernel32" (ByVal hHandle As Long, ByVal dwMilliseconds As Long) As Long
```

```
Sub Main
```

```
Dim hProcess As Long
```

```
Dim TaskId As Long
```

```
    TaskId = Shell("/QDSquidVSM/Tools/ExportData.exe", vbNormalFocus)
```

```
    hProcess = OpenProcess(SYNCHRONIZE, True, TaskId)
```

```
    Call WaitForSingleObject(hProcess, WAIT_INFINITE)
```

```
    CloseHandle hProcess
```

```
End Sub
```



i) Operation of serial com port

- Code written by Juris Prikulis, QD-user in Riga:
- <https://github.com/LZP-2020-1-0200/WinWrap-Basic-SCPI.git>
- SCPI code example provided for Keithley 2400

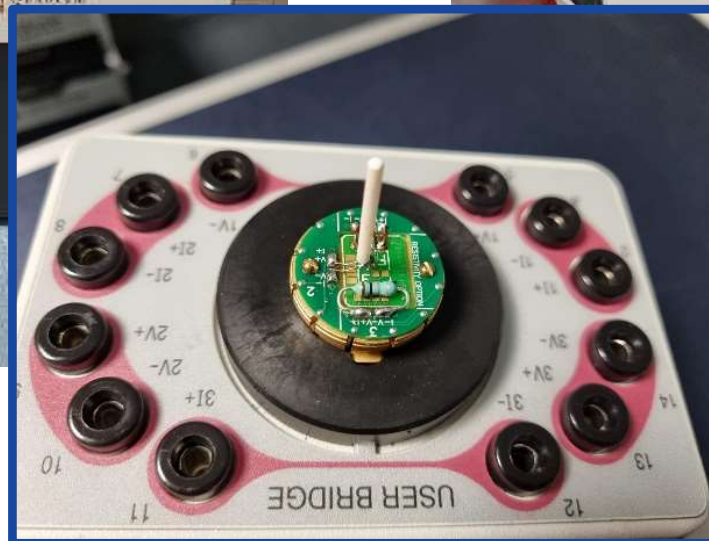


```
'#Language "WWB-COM"  
'#uses "../scpi/scpi.obm"  
'16-Nov-2021 Juris Prikulis  
'https://github.com/LZP-2020-1-0200/WinWrap-Basic-SCPI.git
```

Option Explicit

```
Sub Main  
    Debug.Clear  
    scpi.Open("COM6")  
    scpi.CaptureTimeouts()  
    scpi.SetTimeouts(0,0,10,0,500)  
    scpi.ConfigPort("baud=9600 data=8 parity=n stop=1")  
    scpi.CheckCommState()  
  
    scpi.Write("*IDN?")  
    Wait 0.1  
  
    Dim read_result As String  
    read_result = scpi.Read()  
    Debug.Print read_result  
  
    scpi.Write("CONFigure?")  
    Wait 0.1  
    read_result = scpi.Read()  
    Debug.Print read_result  
  
    scpi.Write(":Output 1")  
    Wait 0.1  
    read_result = scpi.Read()  
    Debug.Print read_result  
  
    scpi.Write(":Read?")  
    Wait 1  
    read_result = scpi.Read()  
    Debug.Print read_result  
  
    scpi.Write(":Output 0")  
    Wait 0.1  
    read_result = scpi.Read()  
    Debug.Print read_result  
  
    scpi.RestoreTimeouts()  
    scpi.Close()  
End Sub
```


j) Lake Shore Model81 SSM & DynaCool



Dynacool

CAN

PC

MultiVu

yyy.bas

M81.obm

scpi.obm

USB
Virtual COM

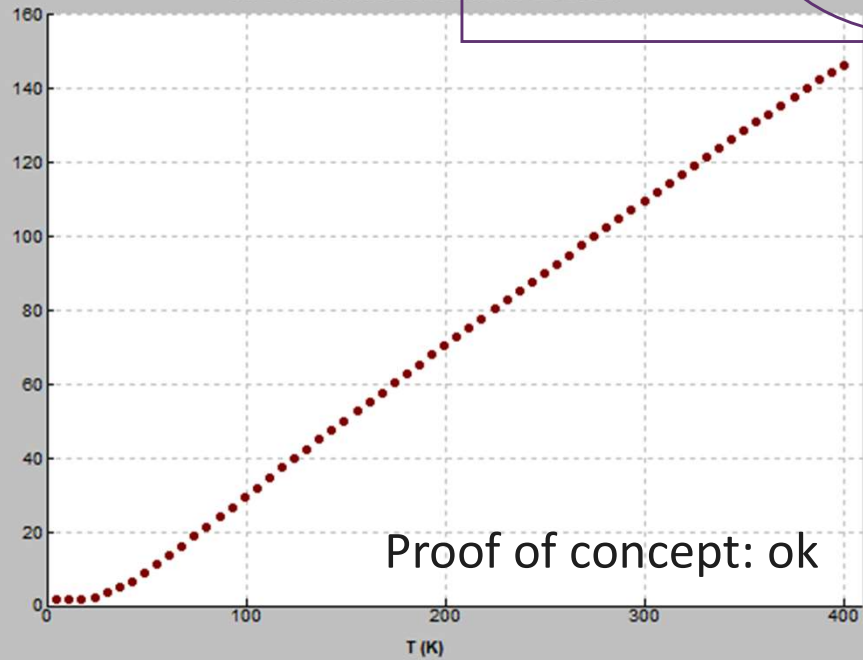
M81 SSM

BCS-10

VM-10

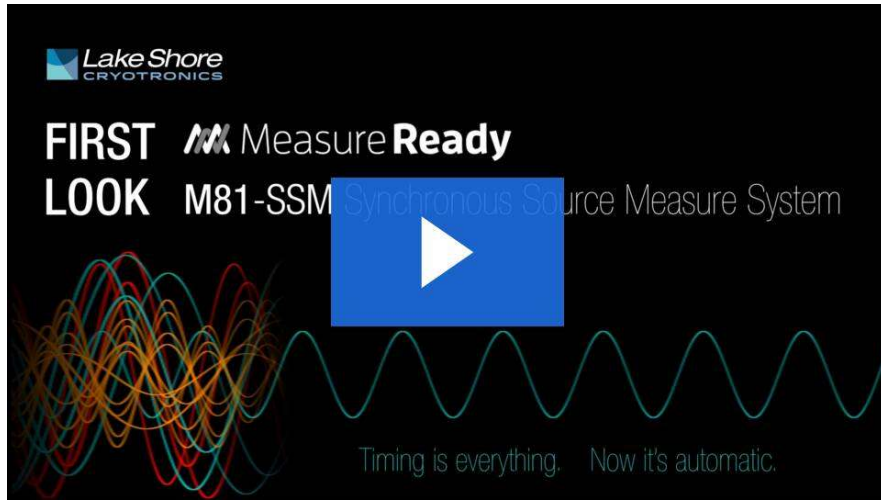
DUT

M81 - 4 Wire Measurement Ohmic LIA Result



Proof of concept: ok



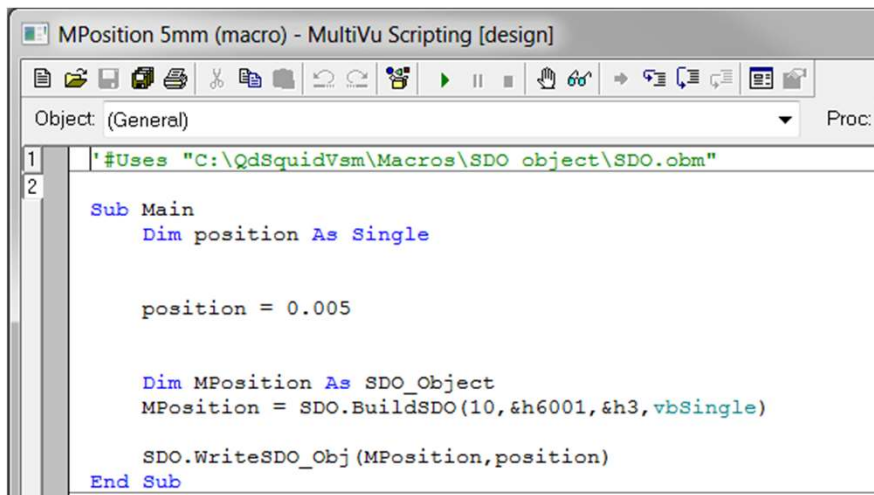


<https://www.lakeshore.com/?wvideo=f3uh432gq1>



Tiny software tools

- Pressure monitor
- VSM-head-mover

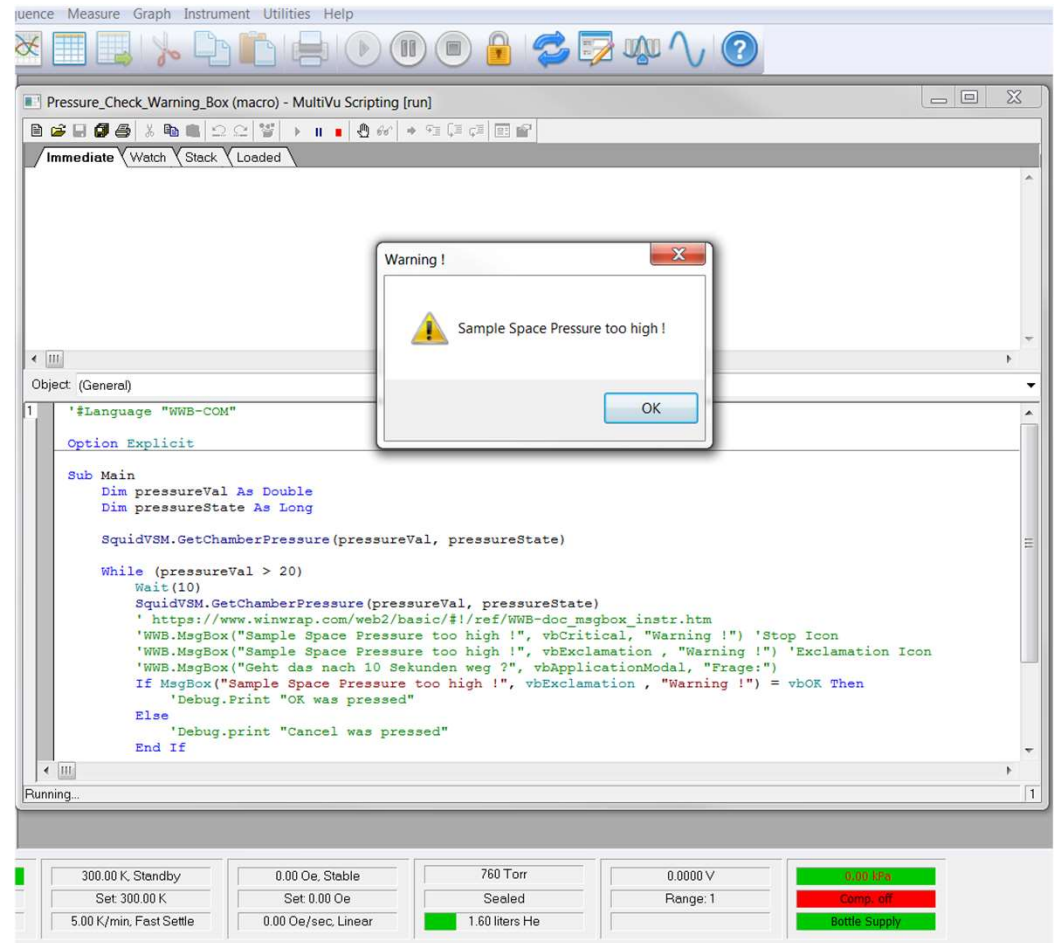


```
1 '#Uses "C:\QdSquidVsm\Macros\SDO object\SDO.obm"
2
Sub Main
    Dim position As Single

    position = 0.005

    Dim MPosition As SDO_Object
    MPosition = SDO.BuildSDO(10, &h6001, &h3, vbSingle)

    SDO.WritesDO_Obj(MPosition, position)
End Sub
```



Warning !
Sample Space Pressure too high !
OK

```
1 '$Language "VBA-COM"
Option Explicit

Sub Main
    Dim pressureVal As Double
    Dim pressureState As Long

    SquidVSM.GetChamberPressure(pressureVal, pressureState)

    While (pressureVal > 20)
        Wait(10)
        SquidVSM.GetChamberPressure(pressureVal, pressureState)
        ' https://www.winwrap.com/web2/basic/#!/ref/WWB-doc_magbox_instr.htm
        ' WVB.MsgBox("Sample Space Pressure too high !", vbCritical, "Warning !") 'Stop Icon
        ' WVB.MsgBox("Sample Space Pressure too high !", vbExclamation, "Warning !") 'Exclamation Icon
        ' WVB.MsgBox("Geht das nach 10 Sekunden weg ?", vbApplicationModal, "Frage:")
        If MsgBox("Sample Space Pressure too high !", vbExclamation, "Warning !") = vbOK Then
            'Debug.Print "OK was pressed"
        Else
            'Debug.print "Cancel was pressed"
        End If
    End While
End Sub
```

300.00 K Standby	0.00 Oe Stable	760 Torr	0.0000 V	Gas On
Set 300.00 K	Set 0.00 Oe	Sealed	Range: 1	Comp. off
5.00 K/min, Fast Settle	0.00 Oe/sec. Linear	1.60 liters He		Bottle Supply

Automated backup to network by (task-)scheduler

- a shell-script (batch-file) leads to below ...

```
C:\WINDOWS\SYSTEM32\cmd.exe
hly_Repeated\ThermalTransport" /S /E /COPY:DAT /PURGE /MIR /R:1000000 /W:30 /LOG+: "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log"

Log File : k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log

C:\WINDOWS\system32>robocopy "c:\QDSquidVSM\SR0M" "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\SR0M" /S /E /COPY:DAT /PURGE /MIR /R:1000000 /W:30 /LOG+: "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log"

Log File : k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log

C:\WINDOWS\system32>robocopy "c:\QDSquidVSM\SVsmULF" "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\SVsmUlf" /S /E /COPY:DAT /PURGE /MIR /R:1000000 /W:30 /LOG+: "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log"

Log File : k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log

C:\WINDOWS\system32>robocopy "c:\QDSquidVSM\Tools" "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\Tools" /S /E /COPY:DAT /PURGE /MIR /R:1000000 /W:30 /LOG+: "k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log"

Log File : k:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\MPMS3-290Backup.log
ECHO is on.
MPMS3 data backed up to K:\QD_SERV\SquidVSM-MPMS3\Darmstadt MPMS3-290\Backup_Monthly_Repeated\
Mon 02/28/2022 12:17:52.51
Skript by stefan riesner
ECHO is on.

C:\WINDOWS\system32>pause
Press any key to continue . . .
```



System warning by mail upon critical state

- Python script to run in parallel to MultiVu
- Script scans (log) file
- Log file entries compared against user defined critical threshold limits
- Warning mail sent to operator upon critical value reached.
 - E.g.: Evercool compressor off
 - Helium level low
 - Supply bottle empty

Diese E-Mail kommt von Deinem **MPMS3**: es leidet !

Systemzeit: 2020-01-24, 09:32:48

Liste der letzten 10 Eintraege bzgl. Parameter: Compressor Status (code)

Time (sec), Compressor Status (code)

3634436960.20, 1
3634436983.04, 1
3634437003.04, 1
3634437015.84, 1
3634437036.51, 1
3634437057.45, 1
3634437078.01, 1
3634437098.81, 1
3634437119.64, 1
3634437140.58, 0

Das *MPMS3* bittet um schnelle *Hilfe* !



Ein höfliches MPMS3 ;-)

```
P:\QD-Apps\Scripts_Python_created_by_LOT\AnalyseLogFileAndWarn.py - Notepad++

Datei Bearbeiten Suchen Ansicht Kodierung Sprachen Einstellungen Werkzeuge Makro Ausführen Erweiterungen Fenster ?

TestAnne.py x io26.gd x SendWarningMail.py x SendWarningMail.py x AnalyseLogFileAndWarn.py x

80
81 if __name__ == "__main__": # Script nicht als Modul geladen
82     FileToSearch = "BRLog.dat" # pure file name
83     #PathToSearchFile = ""
84     PathToSearchFile = "c:/QDSquidVSM/MultiVu/" # slash ok. even in windows. drive letter not required if script and log file on same drive
85     ColumnSearchTerm = "Compressor Status (code)" # exact syntax copied from between commas off header line of data section
86     ColumnAlarmValue = 0.0 # threshold value
87     ColumnAlarmRelation = 'equals' # alternatives 'exceeds' or 'undercuts'
88     DelayTimeBetweenChecks = 120 # seconds
89
90     alarm = False # initialisation. don't chance
91
92     print('Loading Data File to analyse: ' + PathToSearchFile + FileToSearch+'...\n')
93
94
95 while alarm == False:
96     dataFromFile = []
97     #print(alarm) #debug
98     dataFromFile, dataHeader = loadData(PathToSearchFile + FileToSearch)
99     alarm = testForAlarm(dataFromFile, dataHeader, ColumnSearchTerm, ColumnAlarmValue, ColumnAlarmRelation)
100     alarm = testForAlarm(dataFromFile, dataHeader, '2nd stage Therm (K)', 5.0, 'exceeds')
101     alarm = testForAlarm(dataFromFile, dataHeader, '2nd stage Therm (K)', 3.9, 'undercuts')
102     alarm = testForAlarm(dataFromFile, dataHeader, 'EC Dewar Press (kPa)', 4.0, 'exceeds')
103     alarm = testForAlarm(dataFromFile, dataHeader, 'Water In Temp (K)', 26.25, 'exceeds')
104
105     time.sleep(DelayTimeBetweenChecks)
106
```

Python file length: 4,481 lines: 106 Ln: 1 Col: 1 Sel: 0|0 Windows (CR LF) UTF-8 IN

C:\QDCan\CANOpener.exe as an aid to access low level

The screenshot displays the CANOpener application window. The 'File' menu is open, and the 'Export' option is highlighted. Below the menu, a table lists various SDOs. The 'Export' dialog box is open, showing a file explorer view of the 'QMaps' directory. The 'SDO Listings' panel on the right shows a list of SDOs with their attributes, data types, and lengths.

Export

Node	Description	SDO Idx/ PDO Num/ M6000 Ch	SDO Subidx	Type	R/W/ PDO/ PPMS	Read Data	Read Status	Read	Write Data	Write Status
21	Channel 4 Temperature	0x6030	0x1	Float-32	RO			Read		

Export

Local Disk (C:) > This PC > Local Disk (C:) > QdDynacool > QMaps

Name	Date modified	Type	Size
ReadTTO_Thermometers_raw.qmap	3/2/2021 3:24 PM	QMAP File	1 KB
Oven.qmap	9/7/2020 3:54 PM	QMAP File	1 KB
Oven-Thermistor.qmap	9/7/2020 3:49 PM	QMAP File	1 KB
UserTemp-Ch4-BRT.qmap	7/9/2020 9:55 AM	QMAP File	1 KB
BRT_Module_Experimental_SR.qmap	1/16/2020 11:11 AM	QMAP File	1 KB
Full PSU-temperatures.qmap	1/13/2020 3:47 PM	QMAP File	1 KB
Hybrid-fan.qmap	1/13/2020 3:46 PM	QMAP File	1 KB
PressureGauge_Full-NT.qmap	1/23/2019 2:55 PM	QMAP File	1 KB
LightSource.qmap	10/31/2018 10:33 ...	QMAP File	1 KB
TCM-Temps-Readback.qmap	3/13/2018 4:02 PM	QMAP File	1 KB

SDO Listings

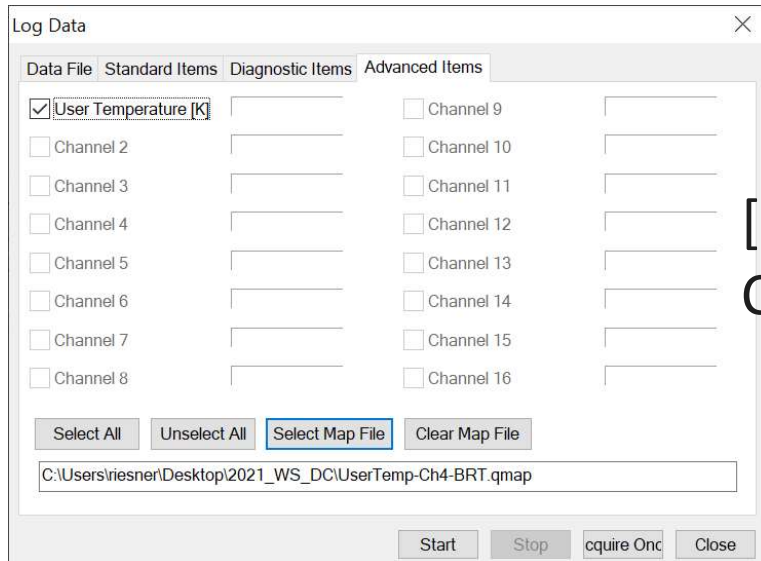
Load from internal cache. May not be up-to-date.

ndx.sub	Attribs	Data Type	Length	Field description
0x6020.1	RW	float	4	Set/Read Coil Current(mA)
0x6020.2	RO	float	4	Coil Compliance Voltage(V)
0x6020.3	RO	float	4	Corrected Coil Current(mA)
0x6020.4	RO	unsigned 16-bit	2	Coil DAC counts
0x6030				Temperature
0x6030.1	RO	float	4	Channel 4 Temperature
0x6800				Table download control
0x6800.1	RW	unsigned 8-bit	1	Table number
0x6800.2	RW	unsigned 32-bit	4	Error code for the last table operation
0x6800.3	RO	string	80	Error string for the last table operation



Qmaps selfmade / made from CANOpener

- Understanding principle of „qmaps“ helps embedding SDOs in your own code

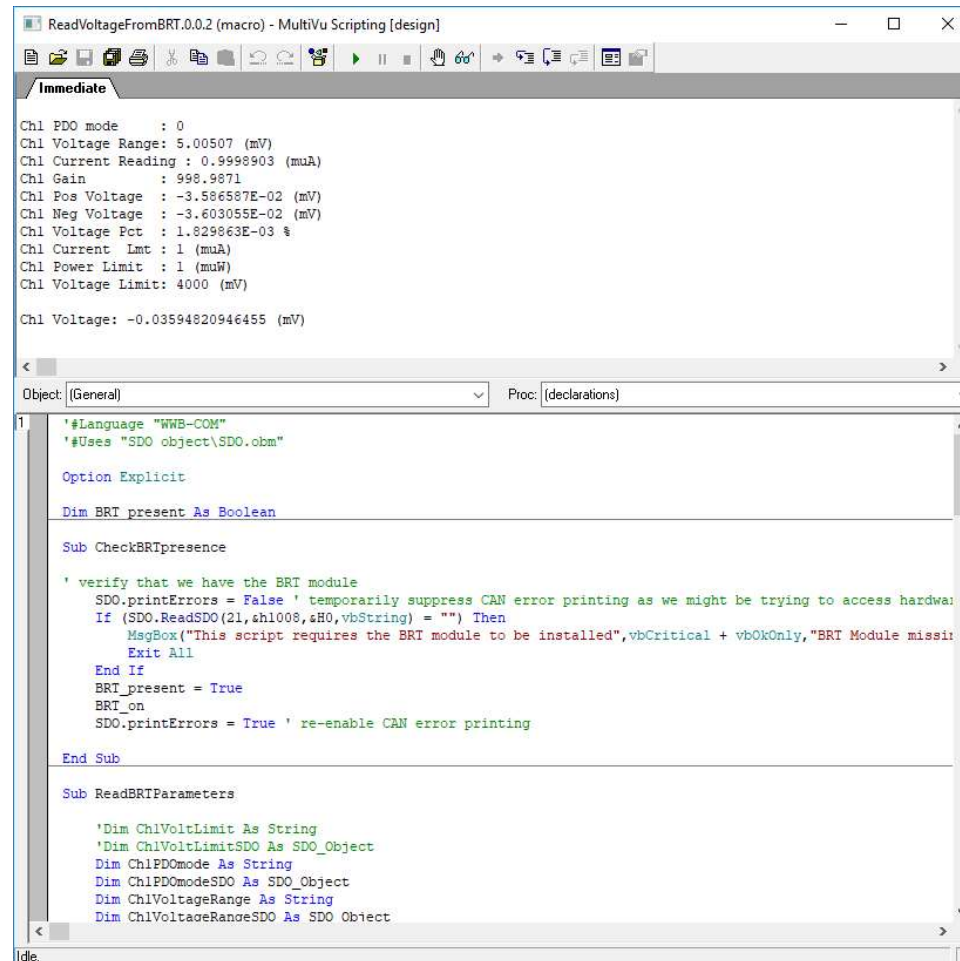


[UserTemp-Ch4-BRT.qmap]

Channel1=21,0x6030,0x1,FLT,User Temperature [K]



CAN-Modules, low level SDO (Service Data Objects) access Read Resistance from BRT Module:



The screenshot shows a MultiVu Scripting window titled "ReadVoltageFromBRT.0.0.2 (macro) - MultiVu Scripting [design]". The window is divided into two main sections: an "Immediate" window at the top and a script editor at the bottom.

The "Immediate" window displays the following data:

```
Chl PDO mode      : 0
Chl Voltage Range: 5.00507 (mV)
Chl Current Reading : 0.9998903 (muA)
Chl Gain          : 998.9871
Chl Pos Voltage   : -3.586587E-02 (mV)
Chl Neg Voltage   : -3.603055E-02 (mV)
Chl Voltage Pct   : 1.829863E-03 %
Chl Current Lmt   : 1 (muA)
Chl Power Limit   : 1 (muW)
Chl Voltage Limit : 4000 (mV)

Chl Voltage: -0.03594820946455 (mV)
```

The script editor shows the following code:

```
1 '#Language "VWB-COM"
  '#Uses "SDO object\SDO.obm"

Option Explicit

Dim BRT_present As Boolean

Sub CheckBRTpresence
    ' verify that we have the BRT module
    SDO.printErrors = False ' temporarily suppress CAN error printing as we might be trying to access hardware
    If (SDO.ReadSDO(21,chl008,$H0,vbString) = "") Then
        MsgBox("This script requires the BRT module to be installed",vbCritical + vbOkOnly,"BRT Module missing")
        Exit All
    End If
    BRT_present = True
    BRT_on
    SDO.printErrors = True ' re-enable CAN error printing
End Sub

Sub ReadBRTParameters
    'Dim ChlVoltLimit As String
    'Dim ChlVoltLimitSDO As SDO_Object
    Dim ChlPDOmode As String
    Dim ChlPDOmodeSDO As SDO_Object
    Dim ChlVoltageRange As String
    Dim ChlVoltageRangeSDO As SDO_Object
```



```
"Language: "WEB.COM"

"Uses: SDO object(SDO_obj)"

Option Explicit

Dim BRT_present As Boolean

Sub CheckBRTpresence

    ' verify that we have the BRT module

    SDO_printErrors = False           ' temporarily suppress CAN error printing as we might be trying to access hardware that is not present

    If (SDO.ReadSDO(21,4h1006,4h0,vbString) = "") Then

        MsgBox("This script requires the BRT module to be installed",vbCritical + vbOkOnly,"BRT Module missing")

        Exit Sub

    End If

    BRT_present = True

    BRT_on

    SDO_printErrors = True 're-enable CAN error printing

End Sub
```

```
Sub ReadBRTParameters

    Dim Ch3ValLimit As String

    Dim Ch3ValLimitSDO As SDO_Object

    Dim Ch3PDOmode As String

    Dim Ch3PDOmodeSDO As SDO_Object

    Dim Ch3VoltageRange As String

    Dim Ch3VoltageRangeSDO As SDO_Object

    Dim Ch3CurrentReading As String

    Dim Ch3CurrentReadingSDO As SDO_Object

    Dim Ch3Gain As String

    Dim Ch3GainSDO As SDO_Object

    Dim Ch3PosVoltage As Single

    Dim Ch3PosVoltageSDO As SDO_Object

    Dim Ch3NegVoltage As Single

    Dim Ch3NegVoltageSDO As SDO_Object

    Dim Ch3VoltagePercentage As String

    Dim Ch3VoltagePercentageSDO As SDO_Object

    Dim Ch3VoltageLimit As String

    Dim Ch3VoltageLimitSDO As SDO_Object

    Dim Ch3PowerLimit As String
```



```
Tot6001_8 FLT,Ch#1 Volt limit (mV)

Tot6001_8_U8,Ch#1 PDD mode

Tot6011_1 FLT,Ch#1 Voltage Range

Tot6001_2 FLT,Ch#1 Current reading (mA)

Tot6011_8 FLT,Ch#1 Gain

Tot6011_8 FLT,Ch#1 Pos Voltage (mV)

Tot6011_9 FLT,Ch#1 Neg Voltage (mV)

Tot6011_A FLT,Ch#1 Volt %

Tot6001_6 FLT,Ch#1 Current Limit (mV)

Tot6001_7 FLT,Ch#1 Power Limit (mW)

Tot6001_8 FLT,Ch#1 Voltage Limit (mA)

Tot6001_3_U8,Ch#1 enable flag. 1 = On, 0 = off

Ch1VoltLimitSDO = SDO.BuHSDO(21,BH6001,BH8)

Ch1PDDmodeSDO = SDO.BuHSDO(21,BH6001,BH8)

Ch1VoltageRangeSDO = SDO.BuHSDO(21,BH6011,BH1)

Ch1CurrentReadingSDO = SDO.BuHSDO(21,BH6001,BH2)

Ch1GainSDO = SDO.BuHSDO(21,BH6011,BH8)

Ch1PosVoltageSDO = SDO.BuHSDO(21,BH6011,BH8)

Ch1NegVoltageSDO = SDO.BuHSDO(21,BH6011,BH9)

Ch1VoltagePercentageSDO = SDO.BuHSDO(21,BH6011,BH4)

Ch1CurrentLimitSDO = SDO.BuHSDO(21,BH6001,BH6)

Ch1PowerLimitSDO = SDO.BuHSDO(21,BH6001,BH7)

Ch1VoltageLimitSDO = SDO.BuHSDO(21,BH6001,BH8)

Ch1NegVoltage = SDO.ReadSDO_Obj(Ch1NegVoltageSDO)

Ch1PosVoltage = SDO.ReadSDO_Obj(Ch1PosVoltageSDO)

Debug.Print

Debug.Print "Ch1 Voltage limit: " & CStr(SDO.ReadSDO_Obj(Ch1VoltLimitSDO)) & " (mV)"

Debug.Print "Ch1 PDD mode : " & CStr(SDO.ReadSDO_Obj(Ch1PDDmodeSDO)) & ""

Debug.Print "Ch1 Voltage Range: " & CStr(SDO.ReadSDO_Obj(Ch1VoltageRangeSDO)) & " (mV)"

Debug.Print "Ch1 Current Reading : " & CStr(SDO.ReadSDO_Obj(Ch1CurrentReadingSDO)) & " (mA)"

Debug.Print "Ch1 Gain : " & CStr(SDO.ReadSDO_Obj(Ch1GainSDO)) & ""

Debug.Print "Ch1 Pos Voltage : " & CStr(SDO.ReadSDO_Obj(Ch1PosVoltageSDO)) & " (mV)"

Debug.Print "Ch1 Neg Voltage : " & CStr(SDO.ReadSDO_Obj(Ch1NegVoltageSDO)) & " (mV)"

Debug.Print "Ch1 Pos Voltage : " & CStr(Ch1PosVoltage) & " (mV)"

Debug.Print "Ch1 Neg Voltage : " & CStr(Ch1NegVoltage) & " (mV)"

Debug.Print "Ch1 Voltage Pct : " & CStr(SDO.ReadSDO_Obj(Ch1VoltagePercentageSDO)) & " %"

Debug.Print "Ch1 Current Limit : " & CStr(SDO.ReadSDO_Obj(Ch1CurrentLimitSDO)) & " (mA)"

Debug.Print "Ch1 Power Limit : " & CStr(SDO.ReadSDO_Obj(Ch1PowerLimitSDO)) & " (mW)"
```

Sub BRT_on

' we put some finite limits for voltage, power, current

Dim Ch1VoltageLimit As Single

Dim Ch3VoltageLimitSDO As SDO_Object

Dim Ch3PowerLimit As Single

Dim Ch3PowerLimitSDO As SDO_Object

Dim Ch3CurrentLimit As Single

Dim Ch3CurrentLimitSDO As SDO_Object

Dim Ch3Enable As Byte 'unsigned integer, UI8

Dim Ch3EnableSDO As SDO_Object

Ch3CurrentLimit = 1.0 'mA

Ch3PowerLimit = 1.0 'mW

Ch3VoltageLimit = 4000.0 'mV

Ch3Enable = 1

Ch3CurrentLimitSDO = SDO.BuildSDO(21,8H4001,8H4)

Ch3PowerLimitSDO = SDO.BuildSDO(21,8H4001,8H7)

Ch3VoltageLimitSDO = SDO.BuildSDO(21,8H4001,8H4)

Ch3EnableSDO = SDO.BuildSDO(21,8H4001,8H3)

SDO.WriteSDO_Obj((Ch3CurrentLimitSDO), Ch3CurrentLimit)

SDO.WriteSDO_Obj((Ch3PowerLimitSDO), Ch3PowerLimit)

SDO.WriteSDO_Obj((Ch3VoltageLimitSDO), Ch3VoltageLimit)

SDO.WriteSDO_Obj(Ch3EnableSDO, Ch3Enable)

End Sub

Sub BRT_off

' we put zero for each: voltage limit, power lim, current limit

Dim Ch1VoltageLimit As Single

Dim Ch3VoltageLimitSDO As SDO_Object

Dim Ch3PowerLimit As Single

Dim Ch3PowerLimitSDO As SDO_Object

Dim Ch3CurrentLimit As Single

Dim Ch3CurrentLimitSDO As SDO_Object

Dim Ch3Enable As Byte 'unsigned integer, UI8

Dim Ch3EnableSDO As SDO_Object

```
Ch1CurrentLimitSDO = SDO.BuildSDO(21,&h6001,&h6)

Ch1PowerLimitSDO = SDO.BuildSDO(21,&h6001,&h7)

Ch1VoltageLimitSDO = SDO.BuildSDO(21,&h6001,&h8)

Ch1EnableSDO = SDO.BuildSDO(21,&h6001,&h3)
```

```
SDO.WriteSDO_Obj(((Ch1CurrentLimitSDO), Ch1CurrentLimit)

SDO.WriteSDO_Obj(((Ch1PowerLimitSDO), Ch1PowerLimit)

SDO.WriteSDO_Obj(((Ch1VoltageLimitSDO), Ch1VoltageLimit)

SDO.WriteSDO_Obj(((Ch1EnableSDO), Ch1Enable)
```

End Sub

Sub Main

```
Debug.Clear

BRT_present = False

CheckBRTpresence

If (BRT_present) Then

    ReadBRTParameters

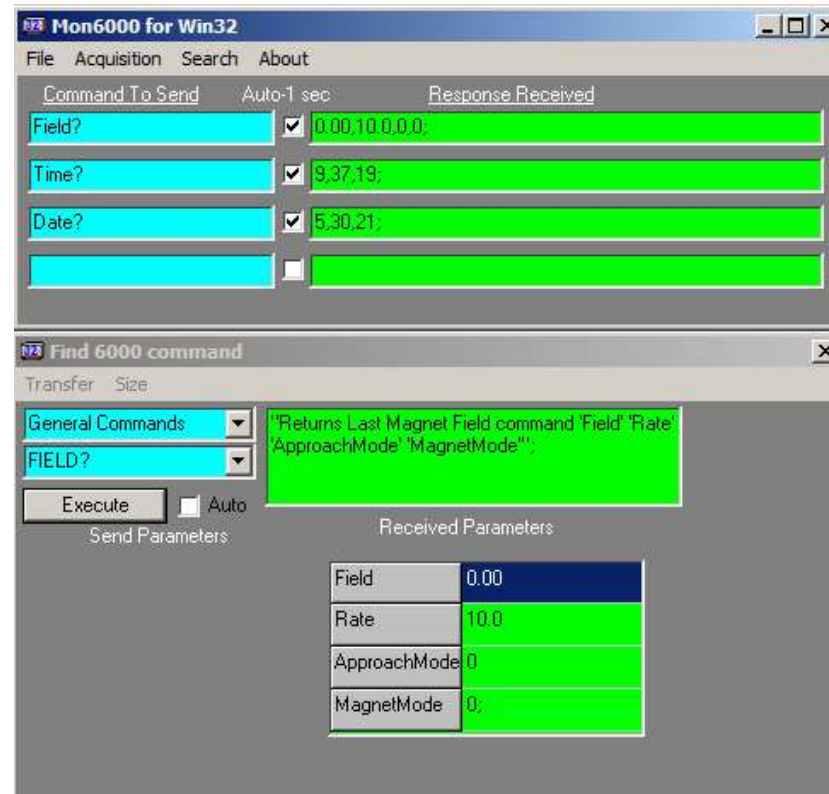
    BRT_off

End If
```

End Sub

C:\QDPpms\Tools\Mon6000.exe as a resource to GPIB-commands understood by M6000

- A PPMS is centrally controlled by a Model 6000 – a GPIB device...



GetDatExample_works_with_PPMS_only.bas

```
'#Language "WWB-COM"
```

```
Option Explicit
```

```
Dim ReadData As String
```

```
Dim ReturnedString As String
```

```
Dim LevelAsNumeric As Double
```

```
Sub Main
```

```
    Debug.Clear
```

```
    PPMS.GetPpmsData(4, ReadData, 0, 2, 0) , we call data from the third page (0,1,_2_ - we count from 0) 3rd position - flag =  $2^{(3-1)}$  = Flag 4
```

```
    ReturnedString = Split(ReadData, ",")(2) 'First(0) parameter: flag, second(1): time stamp, third(2): data
```

```
    Debug.Print "Helium Level: " & ReturnedString
```

```
    LevelAsNumeric = CDbI(ReturnedString)
```

```
    WWB.MsgBox("Helium-Level: " & Format(CDbI(ReturnedString), "#.00") & " %", vbOK, " Helium-Window")
```

```
End Sub
```



Customer set-ups, further reading

- Custom experiment examples from Pharos (<https://www.qdusa.com/pharos/view.php?fDocumentId=2132>)
- Custom experiments from Users in Caen (<https://www.qdusa.com/pharos/view.php?fDocumentId=2782>)
- PPMS 3rd party examples from Barcelona (<https://www.qdusa.com/pharos/view.php?fDocumentId=2781>) - Bernat Bozzo
- Campinas Paper, Brazil: Determination of the magnetocaloric properties using a PPMS (<http://www.qdusa.com/sitedocs/appNotes/ppms/1085-200.pdf>)
- PPMS Vi Library: <https://jrfriedman.people.amherst.edu/ppms-labview.html>
- Python wrapper for the Quantum Design PPMS C# library with remote control: <https://github.com/guenp/PyQDInstrument> (<https://github.com/guenp>)
- https://qcodes.github.io/Qcodes/modules/qcodes/instrument_drivers/QuantumDesign/DynaCoolPPMS/DynaCool.html

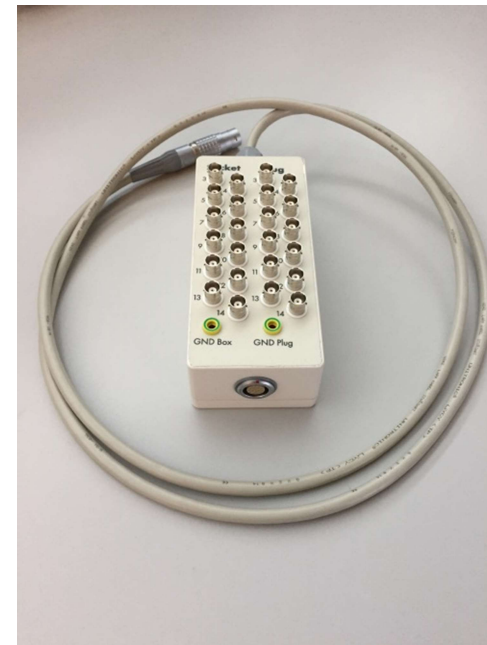


Break out box

- Allows certain „Third Party Concepts“ without special software
- Signal Monitoring from Samples to other third party hardware
- Diagnosis
- Two versions



QD-PBox-L



QD-PBox-K



Quantum Design

EUROPE



| CONFIDENTIAL